



Security Assessment

Mineral Project Coin

Verified on September 24, 2026

SUMMARY

Project

Mineral Project Coin

CHAIN

opBNB

METHODOLOGY

Manual & Automatic Analysis

FILES

Live Contract

DELIVERY

September 24, 2026

TYPE

Standard Audit



11

Total Findings

0

Critical

2

Major

3

Medium

3

Minor

3

Informational

0

Resolved

0 Critical

An exposure that can affect the contract functions in several events that can risk and disrupt the contract

2 Major

An opening & exposure to manipulate the contract in an unwanted manner

3 Medium

An opening that could affect the outcome in executing the contract in a specific situation

3 Minor

An opening but doesn't have an impact on the functionality of the contract

3 Informational

An opening that consists information but will not risk or affect the contract

0 Resolved

ContractWolf's findings has been acknowledged & resolved by the project

STATUS

 **REVIEW REQUIRED**

TABLE OF CONTENTS | Mineral Project Coin

| Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

| Project Information

Token/Project Details
Inheritance Graph
Call Graph

| Findings

Issues
SWC Attacks
CW Assessment
Fixes & Recommendation
Audit Comments

DISCLAIMER | Mineral Project Coin

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

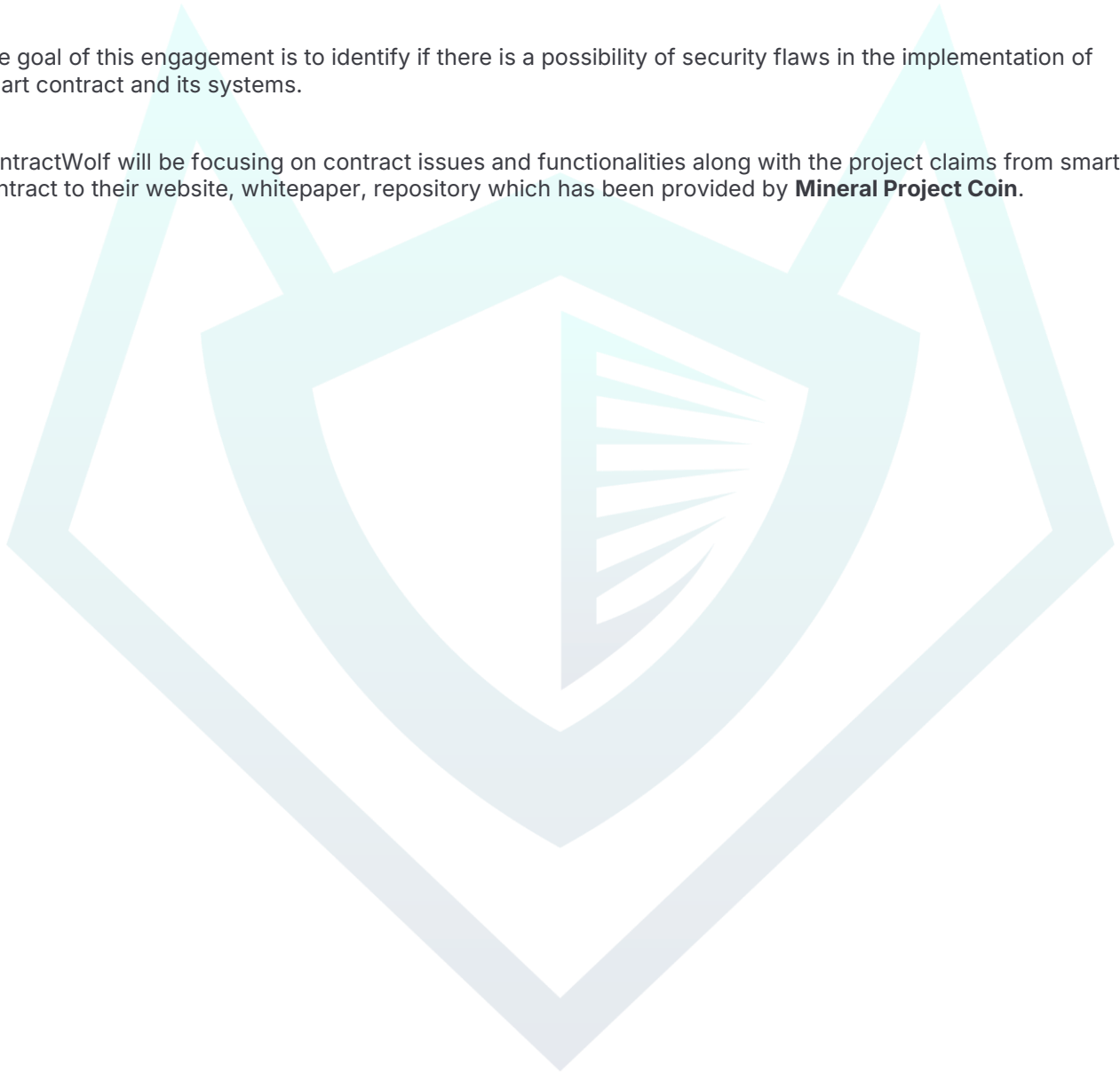
Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | Mineral Project Coin

Mineral Project Coin team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **Mineral Project Coin**.



AUDITING APPROACH | Mineral Project Coin

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
 - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
 - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.
2. Testing and automated analysis that includes :
 - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.
3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.
4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | Mineral Project Coin



Mining RWA Infrastructure, Mineral Project Coin (MPC) is a blockchain-based Real World Asset (RWA) infrastructure designed to standardize the tokenization of mining assets.

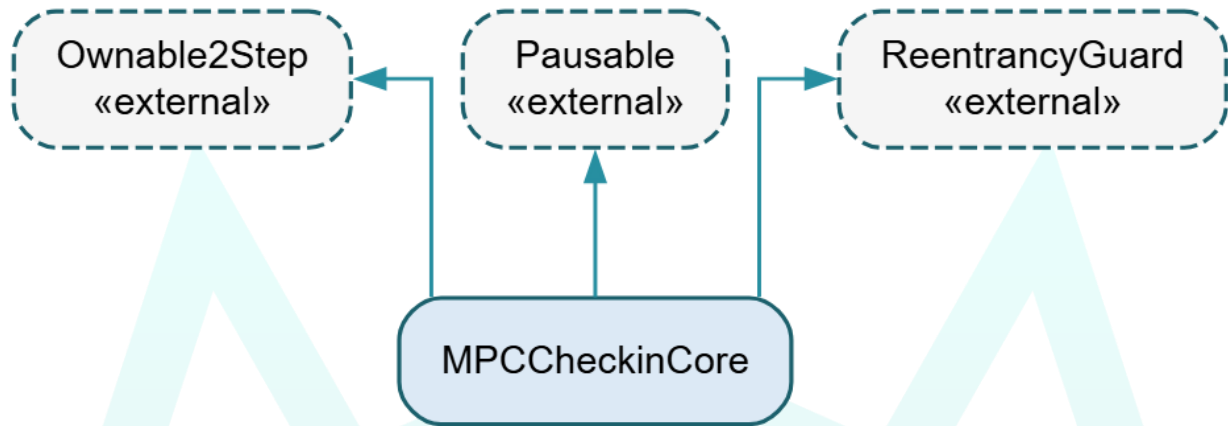
Token Name	Symbol	Decimal	Total Supply	Chain
MPCCheckinCore	MPC	N/A	N/A	opBNB

SOURCE

Source *Live Contract (0x843e4ff9242c3e81b676434b2876fb090d4a5783)*

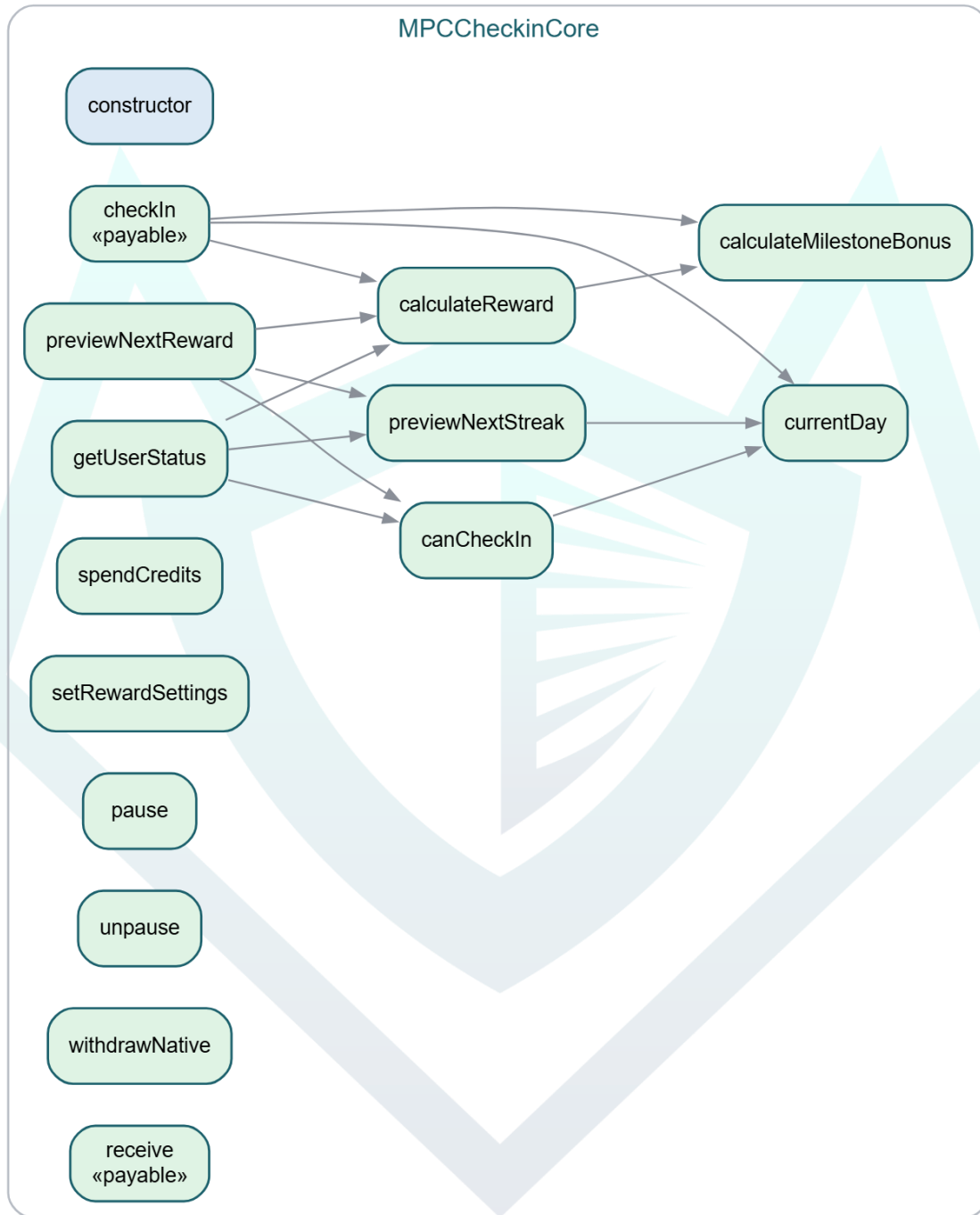
INHERITANCE GRAPH | Mineral Project Coin

Inheritance Graph of Contract Functions



CALL GRAPH | Mineral Project Coin

Call Graph of Contract Functions



FINDINGS | Mineral Project Coin



11

0

2

3

3

3

0

Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

This report has been prepared to state the issues and vulnerabilities for Mineral Project Coin through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
SYBIL-FARM-01	Credit issuance has no Sybil resistance	MPCCheckinCore.sol, L: 164	Major	● Pending
CW-002	Reward parameters are owner-settable with no bounds	MPCCheckinCore.sol, L: 519	Major	● Pending
CW-002 (2)	renounceOwnership() freezes admin functions and strands BNB	MPCCheckinCore.sol, L: 147	Medium	● Pending
SWC-114	Reward settings can be front-run against a pending check-in	MPCCheckinCore.sol, L: 519	Medium	● Pending
CREDIT-REDEEM-01	spendCredits() burns credits with no redemption context and no refund path	MPCCheckinCore.sol, L: 483	Medium	● Pending

SWC-116	Day boundary and streak continuity rest on block.timestamp	MPCCheckinCore.sol, L: 352	Minor	Pending
PAYABLE-CHECKIN-01	checkIn() is payable with no functional use for the value sent	MPCCheckinCore.sol, L: 166	Minor	Pending
EVENT-DOUBLECOUNT-01	StreakBonusAwarded re-reports credits already counted in CheckedIn	MPCCheckinCore.sol, L: 248	Minor	Pending
CW-006	calculateMilestoneBonus() is computed twice on every check-in	MPCCheckinCore.sol, L: 248	Informational	Pending
CW-010	Global and per-user counters share near-identical names	MPCCheckinCore.sol, L: 63	Informational	Pending
PAUSE-PREVIEW-01	View helpers report check-in as available while the contract is paused	MPCCheckinCore.sol, L: 358	Informational	Pending

SWC ATTACKS | Mineral Project Coin

Smart Contract Weakness Classification and Test Cases

ID	Description	Status
SWC-100	Function Default Visibility	● Passed
SWC-101	Integer Overflow and Underflow	● Passed
SWC-102	Outdated Compiler Version	● Passed
SWC-103	Floating Pragma	● Passed
SWC-104	Unchecked Call Return Value	● Passed
SWC-105	Unprotected Ether Withdrawal	● Passed
SWC-106	Unprotected SELF DESTRUCT Instruction	● Passed
SWC-107	Reentrancy	● Passed
SWC-108	State Variable Default Visibility	● Passed
SWC-109	Uninitialized Storage Pointer	● Passed
SWC-110	Assert Violation	● Passed
SWC-111	Use of Deprecated Solidity Functions	● Passed
SWC-112	Delegate call to Untrusted Callee	● Passed
SWC-113	DoS with Failed Call	● Passed
SWC-114	Transaction Order Dependence	● Not Passed
SWC-115	Authorization through tx.origin	● Passed
SWC-116	Block values as a proxy for time	● Not Passed
SWC-117	Signature Malleability	● Passed
SWC-118	Incorrect Constructor Name	● Passed
SWC-119	Shadowing State Variables	● Passed
SWC-120	Weak Sources of Randomness from Chain Attributes	● Passed
SWC-121	Missing Protection against Signature Replay Attacks	● Passed
SWC-122	Lack of Proper Signature Verification	● Passed

ID	Description	Status
SWC-123	Requirement Violation	● Passed
SWC-124	Write to Arbitrary Storage Location	● Passed
SWC-125	Incorrect Inheritance Order	● Passed
SWC-126	Insufficient Gas Griefing	● Passed
SWC-127	Arbitrary Jump with Function Type Variable	● Passed
SWC-128	DoS With Block Gas Limit	● Passed
SWC-129	Typographical Error	● Passed
SWC-130	Right-To-Left-Override control character(U+202E)	● Passed
SWC-131	Presence of unused variables	● Passed
SWC-132	Unexpected Ether balance	● Passed
SWC-133	Hash Collisions With Multiple Variable Arguments	● Passed
SWC-134	Message call with hardcoded gas amount	● Passed
SWC-135	Code With No Effects	● Passed
SWC-136	Unencrypted Private Data On-Chain	● Passed

CW ASSESSMENT | Mineral Project Coin

ContractWolf Vulnerability and Security Tests

ID	Name	Description	Status
CW-001	Multiple Version	Presence of multiple compiler version across all contracts	✓
CW-002	Incorrect Access Control	Additional checks for critical logic and flow	✗
CW-003	Payable Contract	A function to withdraw ether should exist otherwise the ether will be trapped	✓
CW-004	Custom Modifier	major recheck for custom modifier logic	✓
CW-005	Divide Before Multiply	Performing multiplication before division is generally better to avoid loss of precision	✓
CW-006	Multiple Calls	Functions with multiple internal calls	✗
CW-007	Deprecated Keywords	Use of deprecated functions/operators such as block.blockhash() for blockhash(), msg.gas for gasleft(), throw for revert(), sha3() for keccak256(), callcode() for delegatecall(), suicide() for selfdestruct(), constant for view or var for actual type name should be avoided to prevent unintended errors with newer compiler versions	✓
CW-008	Unused Contract	Presence of an unused, unimported or uncalled contract	✓
CW-009	Assembly Usage	Use of EVM assembly is error-prone and should be avoided or double-checked for correctness	✓
CW-010	Similar Variable Names	Variables with similar names could be confused for each other and therefore should be avoided	✗
CW-011	Commented Code	Removal of commented/unused code lines	✓
CW-012	SafeMath Override	SafeMath is no longer needed starting with Solidity v0.8+. The compiler now has built-in overflow checking.	✓

FIXES & RECOMMENDATION

SYBIL-FARM-01 | Credit issuance has no Sybil resistance

`checkIn()` is permissionless and free apart from gas. The only thing standing between a caller and newly issued credits is `hasCheckedInOnDay[msg.sender][today]`, which is keyed on the address. That correctly enforces one check-in per wallet per day, but it places no cost on having more wallets. A single operator can fund thousands of addresses and collect `baseReward` from every one of them each day, and on opBNB the gas for one `checkIn()` is a fraction of a cent, so the marginal cost of minting credits is effectively zero. `totalUsers`, `totalCheckIns` and `totalCreditsIssued` are therefore not trustworthy figures, and any off-chain value ever attached to `availableCredits` is farmable at scale by one party.

```
function checkIn()
    external
    payable
    whenNotPaused
    nonReentrant
    returns (uint256 streak, uint256 credits)
{
    uint256 today = currentDay();

    // The only gate: one per address per day. Addresses are free.
    if (hasCheckedInOnDay[msg.sender][today]) {
        revert AlreadyCheckedInToday();
    }
    ...
}
```

Recommendation

Bind issuance to something the attacker cannot mint for free. The lightest change that actually works is to require a per-wallet signature from a backend key that has already run whatever identity or anti-abuse checks the project uses off-chain, so an address alone is not sufficient. Alternatives, depending on the product: gate on a non-transferable badge, or charge a small `msg.value` sized so that farming costs more than the credits are worth. Whichever route is taken, treat `totalUsers` as an upper bound rather than a real user count until it is enforced.

```
address public signerAuthority;
mapping(address => uint256) public checkInNonce;

error InvalidSignature();

function checkIn(bytes calldata signature)
    external
    whenNotPaused
    nonReentrant
    returns (uint256 streak, uint256 credits)
{
    uint256 today = currentDay();

    if (hasCheckedInOnDay[msg.sender][today]) {
        revert AlreadyCheckedInToday();
    }

    bytes32 digest = MessageHashUtils.toEthSignedMessageHash(
        keccak256(
            abi.encode(
                block.chainid,
                address(this),
                msg.sender,
                today,
                checkInNonce[msg.sender]++
            )
        )
    );

    if (ECDSA.recover(digest, signature) != signerAuthority) {
        revert InvalidSignature();
    }

    // ... unchanged streak and credit accounting ...
}
```

CW-002 | Reward parameters are owner-settable with no bounds

`setRewardSettings()` validates only that `newWeeklyBonusThreshold` and `newMonthlyBonusThreshold` are non-zero. The other four values are written straight to storage unchecked, and the function has no timelock. Two concrete consequences follow. Setting `streakRewardIncrement` to a very large value makes `baseReward + ((streak - 1) * streakRewardIncrement)` inside `calculateReward()` overflow; under Solidity 0.8.x that reverts rather than wrapping, so `checkIn()` becomes uncallable for every wallet whose streak has reached 2 — a denial of service on the contract's only entry point that persists until the owner undoes it. Separately, `newWeeklyBonusThreshold` of 1 makes `streak % weeklyBonusThreshold == 0` true on every check-in, paying `weeklyBonus` daily to everyone.

```
function setRewardSettings(
    uint256 newBaseReward,
    uint256 newStreakRewardIncrement,
    uint256 newWeeklyBonusThreshold,
    uint256 newMonthlyBonusThreshold,
    uint256 newWeeklyBonus,
    uint256 newMonthlyBonus
)
    external
    onlyOwner
{
    if (newWeeklyBonusThreshold == 0 || newMonthlyBonusThreshold == 0) {
        revert InvalidRewardConfiguration();
    }

    baseReward = newBaseReward; // unbounded
    streakRewardIncrement = newStreakRewardIncrement; // unbounded -> can brick checkIn()
    weeklyBonusThreshold = newWeeklyBonusThreshold; // may be 1
    monthlyBonusThreshold = newMonthlyBonusThreshold; // may be 1
    weeklyBonus = newWeeklyBonus; // unbounded
    monthlyBonus = newMonthlyBonus; // unbounded
    ...
}
```

Recommendation

Put hard ceilings in the contract so no owner key, compromised or not, can push the reward maths into a reverting range or turn the weekly bonus into a per-day payout. Constants are preferable to owner-settable maxima, since a settable ceiling is no ceiling. Require both thresholds to be at least 2 and cap each credit value at a figure the project is willing to commit to publicly.

```
uint256 public constant MAX_BASE_REWARD = 1_000;
uint256 public constant MAX_STREAK_INCREMENT = 100;
uint256 public constant MAX_BONUS = 10_000;
uint256 public constant MIN_THRESHOLD = 2;

function setRewardSettings(
    uint256 newBaseReward,
    uint256 newStreakRewardIncrement,
    uint256 newWeeklyBonusThreshold,
    uint256 newMonthlyBonusThreshold,
    uint256 newWeeklyBonus,
    uint256 newMonthlyBonus
)
    external
    onlyOwner
{
    if (
        newWeeklyBonusThreshold < MIN_THRESHOLD ||
        newMonthlyBonusThreshold < MIN_THRESHOLD ||
        newBaseReward > MAX_BASE_REWARD ||
        newStreakRewardIncrement > MAX_STREAK_INCREMENT ||
        newWeeklyBonus > MAX_BONUS ||
        newMonthlyBonus > MAX_BONUS
    ) {
        revert InvalidRewardConfiguration();
    }

    // ... assignments unchanged ...
}
```

CW-002 (2) | renounceOwnership() freezes admin functions and strands BNB

Ownable2Step hardens `transferOwnership()` into a two-step handshake so ownership cannot be sent to an address that never confirms, but it does not override `renounceOwnership()`. That function is inherited from `Ownable` unchanged and sets `owner()` to `address(0)` in a single unconfirmed call. The careful two-step guard on one path sits next to an unguarded one-way door on the other. Once renounced, `pause()`, `unpause()`, `setRewardSettings()` and `withdrawNative()` can never be called again: reward values freeze permanently, the emergency stop is gone, and every wei the contract has accepted through `receive()` or a payable `checkIn()` is stranded with no recovery path.

```
contract MPCCheckinCore is
    Ownable2Step,      // hardens transferOwnership, but NOT renounceOwnership
    Pausable,
    ReentrancyGuard
{
    constructor() Ownable(msg.sender) {}

    // renounceOwnership() is inherited from Ownable and is a single,
    // unconfirmed call that sets owner() to address(0) forever.
}
```

Recommendation

Disable `renounceOwnership()` outright unless the project genuinely intends to make the contract ownerless. If ownerless operation is the goal, it still should not be reachable while the contract can hold BNB - drain first, and require an explicit confirmation argument so it cannot happen through a mistyped call.

```
error RenounceDisabled();

/// @notice Ownership cannot be renounced: doing so would permanently
///         freeze pause control, reward settings and BNB withdrawal.
function renounceOwnership() public view override onlyOwner {
    revert RenounceDisabled();
}
```

SWC-114 | Reward settings can be front-run against a pending check-in

The credits a wallet receives are read from mutable storage at execution time inside `calculateReward()`, not fixed at the moment the user signs. A user can read 72 from `previewNextReward()`, submit `checkIn()`, and have the owner's `setRewardSettings()` land first in the same block, dropping `baseReward` and `weeklyBonus` to 0 and burning the user's once-per-day slot for almost nothing. The user's transaction still succeeds, so there is no revert to warn them, and `checkIn()` accepts no minimum, so there is no way to express the price they agreed to. This is the ordering dependence SWC-114 describes, with the reward-changing variant rather than the approval one.

```
// Reward is resolved against live storage, at execution time:
uint256 earnedCredits = calculateReward(newStreak);

// ...and setRewardSettings() may be mined immediately before it:
function setRewardSettings(...) external onlyOwner {
    baseReward = newBaseReward;
    weeklyBonus = newWeeklyBonus;
    ...
}
```

Recommendation

Let the caller state the minimum they will accept, so a reward cut mined ahead of them reverts their transaction instead of silently consuming their daily slot. Pair that with a timelock or an announced delay on `setRewardSettings()` so changes are visible before they bind. The two fixes address different halves of the problem and are worth having together.

```
error RewardBelowMinimum(uint256 actual, uint256 minimum);

function checkIn(uint256 minCredits)
    external
    whenNotPaused
    nonReentrant
    returns (uint256 streak, uint256 credits)
{
    // ... streak calculation unchanged ...

    uint256 earnedCredits = calculateReward(newStreak);

    if (earnedCredits < minCredits) {
        revert RewardBelowMinimum(earnedCredits, minCredits);
    }

    // ... accounting unchanged ...
}
```

CREDIT-REDEEM-01 | spendCredits() burns credits with no redemption context and no refund path

`spendCredits()` takes an amount and nothing else. It debits `user.availableCredits` and emits `CreditsSpent(msg.sender, amount, remainingCredits)`, which records no item, order reference or recipient. A backend watching that event cannot tell what the user was redeeming, and cannot distinguish two identical spends by the same wallet in the same block, so it has nothing durable to key idempotent fulfilment on. The gap matters because the burn is irreversible: nothing in the contract increases `availableCredits` except `checkIn()`, so if off-chain fulfilment fails after the transaction confirms, the credits are gone and neither the user nor the owner can restore them.

```
function spendCredits(uint256 amount)
    external
    whenNotPaused
    nonReentrant
{
    if (amount == 0) {
        revert ZeroAmount();
    }

    UserInfo storage user = users[msg.sender];

    if (user.availableCredits < amount) {
        revert InsufficientCredits();
    }

    user.availableCredits -= amount;

    // No reference to what was bought; no way to undo.
    emit CreditsSpent(msg.sender, amount, user.availableCredits);
}
```

Recommendation

Carry an opaque reference through the call and index it on the event, so the backend can match a burn to the order that caused it and reject a replay of one it has already fulfilled. Add an owner-only `refundCredits()` for the failed-fulfilment case; it is a privileged mint of an internal, non-transferable number, so bound it and emit an event rather than leaving operations with no remedy at all.

```
event CreditsSpent(
    address indexed user,
    bytes32 indexed reference,
    uint256 amount,
    uint256 remainingCredits
);

event CreditsRefunded(
    address indexed user,
    bytes32 indexed reference,
    uint256 amount
);

function spendCredits(uint256 amount, bytes32 reference)
    external
    whenNotPaused
    nonReentrant
{
    if (amount == 0) {
        revert ZeroAmount();
    }

    UserInfo storage user = users[msg.sender];

    if (user.availableCredits < amount) {
        revert InsufficientCredits();
    }

    user.availableCredits -= amount;

    emit CreditsSpent(msg.sender, reference, amount, user.availableCredits);
}

function refundCredits(address account, uint256 amount, bytes32 reference)
    external
    onlyOwner
{
    if (amount == 0) {
        revert ZeroAmount();
    }

    users[account].availableCredits += amount;

    emit CreditsRefunded(account, reference, amount);
}
```

SWC-116 | Day boundary and streak continuity rest on block.timestamp

`currentDay()` returns `block.timestamp / 1 days`, so eligibility, the `hasCheckedInOnDay` key and the `user.lastCheckInDay + 1 == today` streak test all derive from a value the block producer chooses. On opBNB the sequencer sets that timestamp, and within the drift the chain tolerates a block near UTC midnight can fall on either side of the boundary. The direct impact is small, a check-in lands a day early or late — but the streak test is exact, so a single misplaced boundary resets a long streak to 1 and costs the user their weekly and monthly milestones.

```
function currentDay() public view returns (uint256) {
    return block.timestamp / 1 days;
}

// Exact equality: one shifted boundary resets the streak to 1.
} else if (user.lastCheckInDay + 1 == today) {
    newStreak = user.currentStreak + 1;
} else {
    newStreak = 1;
}
```

Recommendation

There is no better clock available on-chain, so keep the timestamp-derived day but stop making streak continuity depend on it exactly. Treating a gap of one day as continuing the streak absorbs a boundary shift without letting a genuinely missed day through, since `hasCheckedInOnDay` still caps issuance at one per day.

```
uint256 constant STREAK_GRACE_DAYS = 1;

// Continue the streak when the gap is within the grace window,
// so a shifted day boundary cannot destroy a long streak.
uint256 gap = today - user.lastCheckInDay;

if (user.totalCheckIns == 0) {
    newStreak = 1;
} else if (gap <= 1 + STREAK_GRACE_DAYS) {
    newStreak = user.currentStreak + 1;
} else {
    newStreak = 1;
}
```

PAYABLE-CHECKIN-01 | `checkIn()` is payable with no functional use for the value sent

`checkIn()` is declared payable, but nothing in it prices, refunds or credits `msg.value`, the value is absorbed into the contract balance and the only trace is a `NativeReceived` event. The contract's own documentation states BNB is not required, so any value that arrives here is a mistake: a mis-built front end, or a wallet that pads value on a retry. Because the function accepts it silently, the transaction succeeds and the user gets no error; recovery is then possible only through the owner's `withdrawNative()`, and there is no path that returns it to the sender.

```
function checkIn()
  external
  payable          // accepts BNB the contract has no use for
  whenNotPaused
  nonReentrant
  returns (uint256 streak, uint256 credits)
{
  ...
  // Value is logged and kept; never priced, never refunded.
  if (msg.value > 0) {
    emit NativeReceived(msg.sender, msg.value);
  }
}
```

Recommendation

Drop payable and the `msg.value` block. A value-bearing call then reverts at the compiler-inserted check and the user keeps their BNB, which is the correct outcome for a function that is documented as free. Deliberate donations still have a home in `receive()`, which already logs them.

```
function checkIn()
  external
  whenNotPaused
  nonReentrant
  returns (uint256 streak, uint256 credits)
{
  // No `payable`: a call carrying value now reverts and the
  // sender keeps their BNB. Donations still go through receive().
  ...
}
```

EVENT-DOUBLECOUNT-01 | StreakBonusAwarded re-reports credits already counted in CheckedIn

`calculateReward()` already folds `calculateMilestoneBonus(streak)` into its return value, so the `credits` field of `CheckedIn` is the full figure including any weekly or monthly bonus. `checkIn()` then recomputes that same bonus and emits `StreakBonusAwarded` carrying it a second time. On-chain accounting is correct, `user.totalCredits` and `totalCreditsIssued` are each credited once, but nothing in the two event signatures says the bonus is a subset of `credits` rather than an addition to it. An indexer summing both, which is the natural reading, overstates issuance on every 7th and 30th day of a streak.

```
// calculateReward() already includes the milestone bonus:
uint256 reward = baseReward + ((streak - 1) * streakRewardIncrement);
reward += calculateMilestoneBonus(streak);

// ...so `earnedCredits` here is the full amount, bonus included:
emit CheckedIn(msg.sender, today, newStreak, earnedCredits);

// ...and this emits the same bonus a second time:
uint256 milestoneBonus = calculateMilestoneBonus(newStreak);

if (milestoneBonus > 0) {
    emit StreakBonusAwarded(msg.sender, today, newStreak, milestoneBonus);
}
```

Recommendation

Make the two figures disjoint so any consumer can add them safely. Emitting the base and the bonus as separate fields on `CheckedIn` removes the ambiguity at the source and lets `StreakBonusAwarded` stay as a milestone marker rather than a second ledger entry.

```
event CheckedIn(
    address indexed user,
    uint256 indexed dayIndex,
    uint256 streak,
    uint256 baseCredits,    // excludes the milestone bonus
    uint256 bonusCredits   // the milestone bonus, counted once
);

(uint256 baseCredits, uint256 bonusCredits) = _splitReward(newStreak);
uint256 earnedCredits = baseCredits + bonusCredits;

emit CheckedIn(msg.sender, today, newStreak, baseCredits, bonusCredits);

if (bonusCredits > 0) {
    emit StreakBonusAwarded(msg.sender, today, newStreak, bonusCredits);
}
```

CW-006 | calculateMilestoneBonus() is computed twice on every check-in

checkIn() calls calculateReward(), which internally calls calculateMilestoneBonus(streak), and then calls calculateMilestoneBonus(newStreak) again with the same argument purely to decide whether to emit StreakBonusAwarded. Both invocations read weeklyBonusThreshold, weeklyBonus, monthlyBonusThreshold and monthlyBonus, so the duplicate costs four extra warm SLOAD operations plus two modulo operations on the contract's only hot path, a cost paid by every user on every check-in, forever.

```
// First computation, inside calculateReward():
uint256 earnedCredits = calculateReward(newStreak);

// ... 40 lines later, the identical call again:
uint256 milestoneBonus = calculateMilestoneBonus(newStreak);

if (milestoneBonus > 0) {
    emit StreakBonusAwarded(msg.sender, today, newStreak, milestoneBonus);
}
```

Recommendation

Compute the split once and reuse both halves. An internal helper returning the base and the bonus separately removes the duplicate SLOADs and, as a side effect, gives the event fix in EVENT-DOUBLECOUNT-01 the values it needs.

```
function _splitReward(uint256 streak)
    internal
    view
    returns (uint256 baseCredits, uint256 bonusCredits)
{
    if (streak == 0) {
        return (0, 0);
    }

    baseCredits = baseReward + ((streak - 1) * streakRewardIncrement);
    bonusCredits = calculateMilestoneBonus(streak);
}

// calculateReward() stays as the public view API:
function calculateReward(uint256 streak) public view returns (uint256) {
    (uint256 baseCredits, uint256 bonusCredits) = _splitReward(streak);
    return baseCredits + bonusCredits;
}
```

CW-010 | Global and per-user counters share near-identical names

`totalCheckIns` exists twice with different meanings, once as a global state variable and once as `UserInfo.totalCheckIns`, and `getUserStatus()` has to rename its own return value to `userTotalCheckIns` to work around the collision. The per-user `totalCredits` sits next to the global `totalCreditsIssued`. Inside `checkIn()` the lines `user.totalCheckIns += 1;` and `totalCheckIns += 1;` differ only by the receiver, and they are twelve lines apart. Transposing them would leave the global tally wrong while also breaking the `user.totalCheckIns == 0` test that both the first-time-user branch and the streak reset depend on — a change that compiles cleanly and would not show up until the numbers were audited.

```
uint256 public totalUsers;
uint256 public totalCheckIns;          // global
uint256 public totalCreditsIssued;     // global

struct UserInfo {
    ...
    uint256 totalCheckIns;              // per-user, same name
    uint256 totalCredits;               // near-twin of totalCreditsIssued
    uint256 availableCredits;
}

// Twelve lines apart inside checkIn(), differing only by receiver:
user.totalCheckIns += 1;
totalCheckIns += 1;
```

Recommendation

Give the globals a prefix so no identifier means two things in one contract. The struct fields are already namespaced by `user.`, so renaming the globals is the smaller change and it lets `getUserStatus()` return `totalCheckIns` under its natural name.

```
uint256 public globalTotalUsers;
uint256 public globalTotalCheckIns;
uint256 public globalCreditsIssued;

struct UserInfo {
    ...
    uint256 totalCheckIns;              // now unambiguous
    uint256 lifetimeCredits;           // clearly distinct from the global
    uint256 availableCredits;
}
```

PAUSE-PREVIEW-01 | View helpers report check-in as available while the contract is paused

`canCheckIn()` returns `true` and `previewNextReward()` returns a non-zero figure while the contract is paused, because neither consults `paused()`. `checkIn()` carries `whenNotPaused` and reverts. A front end driving its button off `canCheckIn()` walks the user into a transaction that always fails, and `getUserStatus()` propagates the same wrong value through its `canCheckInToday` field into anything built on that API. The inconsistency shows up precisely during an incident, when the pause is active and clear signalling matters most.

```
function canCheckIn(address account) public view returns (bool) {  
    // Ignores paused(); checkIn() carries whenNotPaused and will revert.  
    return !hasCheckedInOnDay[account][currentDay()];  
}
```

Recommendation

Fold the pause state into `canCheckIn()` so it answers the question callers are actually asking — whether a check-in would succeed right now. `previewNextReward()` and `getUserStatus()` already route through it, so both become correct with no further change.

```
function canCheckIn(address account) public view returns (bool) {  
    if (paused()) {  
        return false;  
    }  
  
    return !hasCheckedInOnDay[account][currentDay()];  
}
```

AUDIT COMMENTS | Mineral Project Coin

Smart Contract audit comment for a non-technical perspective

- Checking in is free, the contract never charges the user, only network gas.
- Credits are not a token. They cannot be transferred, sold or traded, and exist only as numbers inside this contract.
- The owner cannot take, move or reduce any user's credits, no function touches another wallet's balance.
- There is no blacklist. The owner cannot block an individual wallet from checking in.
- Ownership transfer is two-step: a new owner must accept before it takes effect, so the contract cannot be handed to a mistyped address.
- The owner can pause the contract at any time, which stops every check-in and every credit spend until they unpause it.
- The owner can change every reward value at any time, with no upper limit and no delay, including cutting rewards to zero moments before a user's check-in confirms.
- The owner can withdraw all BNB held by the contract to any address of their choosing.
- The owner can permanently renounce ownership, which would freeze the reward settings, remove the pause switch and lock any BNB in the contract forever.
- Anyone can run unlimited wallets and check in from each one, so the user counts and credit totals this contract reports can be inflated cheaply by a single party.
- Spending credits is irreversible and records no reference to what was redeemed; if delivery fails afterwards, the contract has no way to give them back.



CONTRACTWOLF

Blockchain Security - Smart Contract Audits